

FROM ZERO TO PRODUCT: SHIPPING A CUDA APPLIANCE WITH NIXOS

Jacek Galowicz

NixCon 2026 Workshop / 2026-09-26



About Jacek Galowicz

CEO, founder of the Applicative Systems Group

- ▶ M.Sc. EE · 20+ years of low-level software engineering
- ▶ Author of the Nix Antipatterns Book, 8 Nix(OS) articles for heise+ / c't, C++17 STL Cookbook
- ▶ Maintainer of the NixOS integration test driver



FH MÜNSTER
University of Applied Sciences



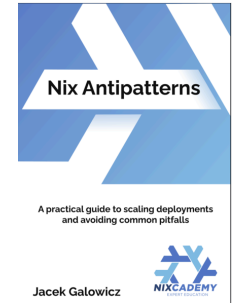
galowicz.de



tfc



jgalowicz



Who we are

Nix(OS) consulting & training

- ▶ **Embedded appliances & OTA updates**
from Yocto/Ubuntu images to NixOS
- ▶ **Reproducible builds & supply chain**
SBOMs & provenance for CRA / NIS2
- ▶ **Nix(OS) training (Nixcademy)**
1000+ trained professionals

applicative.systems · nixcademy.com



Today

Part 1: CUDA app on NVIDIA Jetson

- Initial firmware upgrades
- jetpack-nixos
- CUDA in nixpkgs
- maximum inference FPS app
- size and (boot) speed optimization

Part 2: A/B immutable parts. w/ OTA

- self-growing disk images
- NixOS image size reduction
- immutable A/B partitions
- OTA with systemd-sysupdate
- delta-upgrades with Rugix
- cross-compilation

PART 1

**CUDA JUMP GAME ON THE
JETSON**

The goal: a jump game on the Jetson



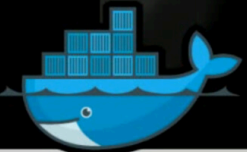
Jump and duck



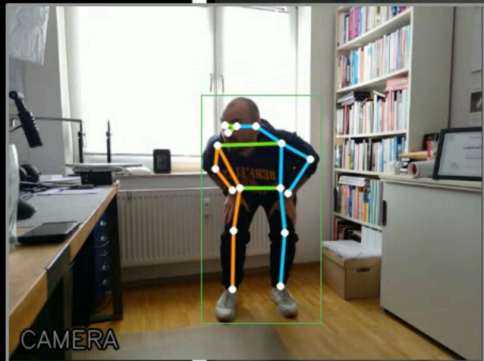
CAMERA



JUMP



50 fps yolo 60 fps loop 16 dropped cpu 6



CAMERA

DUCK



50 fps yolo 60 fps loop 18 dropped cpu 6



CAMERA



JUMP

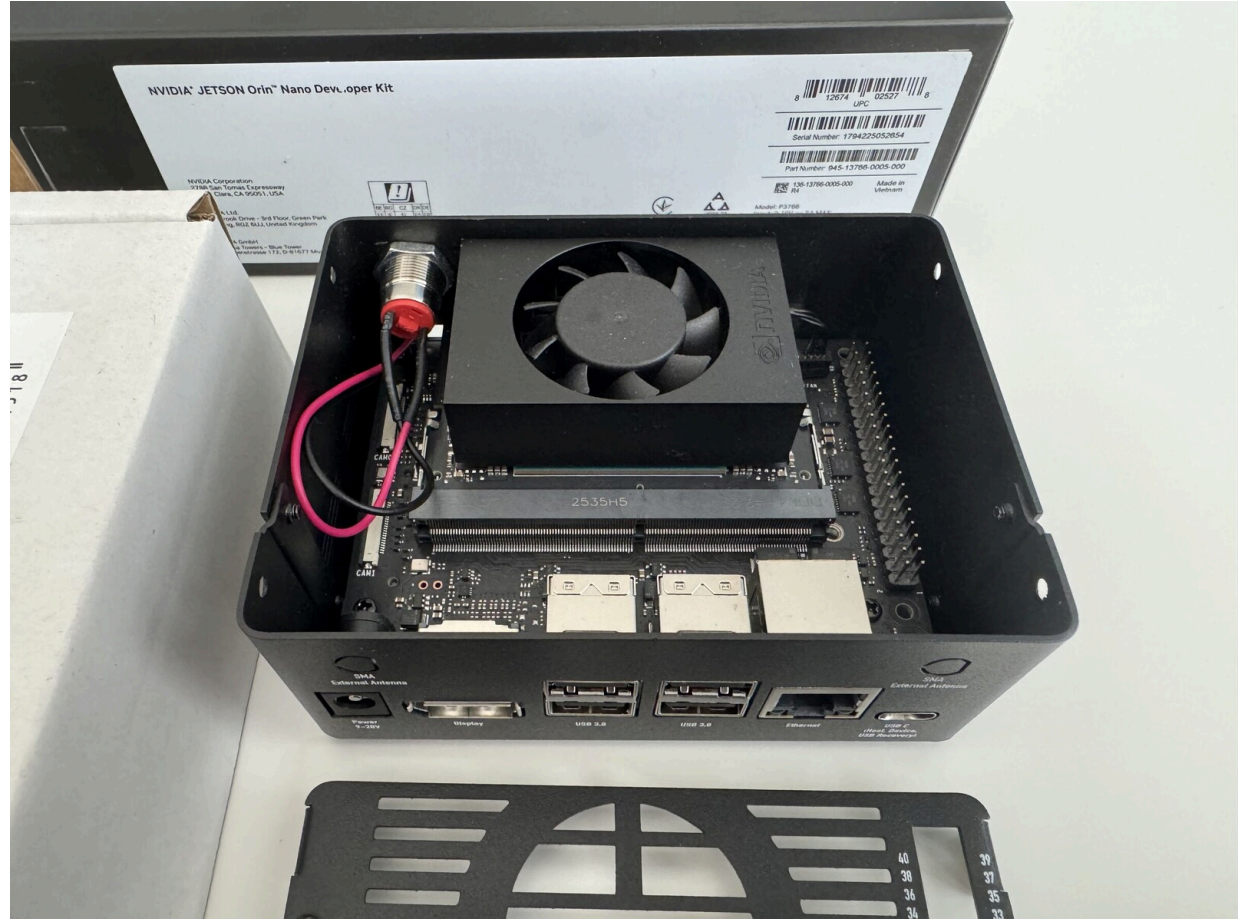


50 fps yolo 60 fps loop 16 dropped cpu 6

Game over



Unpacking the Jetson



The board

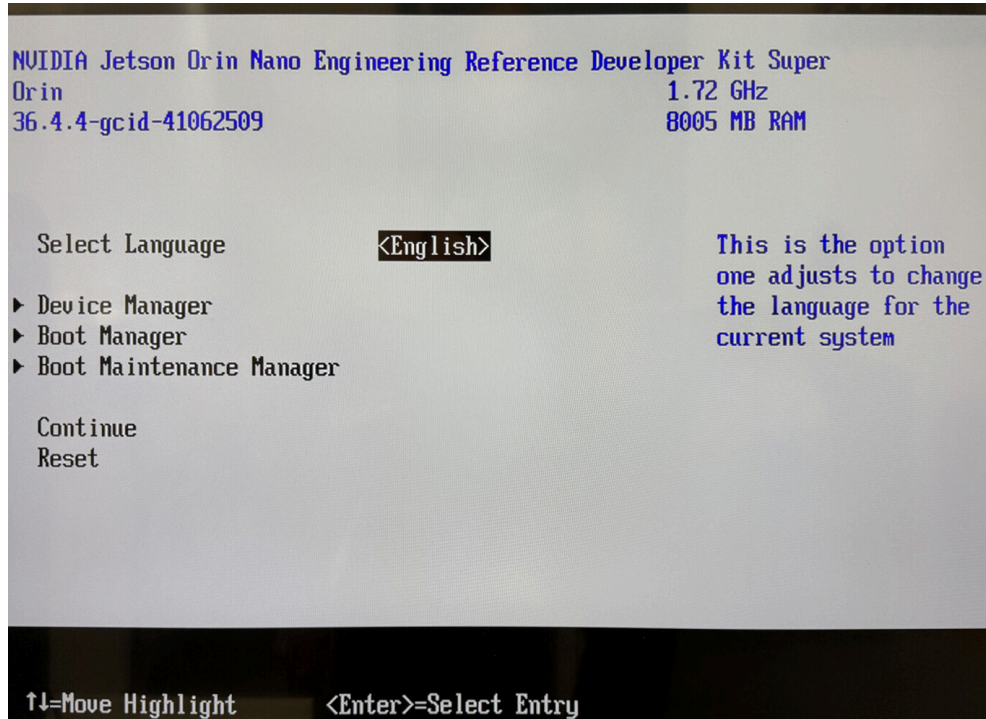
Jetson Orin Nano Super devkit

- aarch64, 8 GB RAM
- GPU: CUDA capability **8.7** (ga10b)
- M.2 Wi-Fi (Realtek RTL8822CE)
- M.2 NVMe slot
- UEFI firmware on QSPI

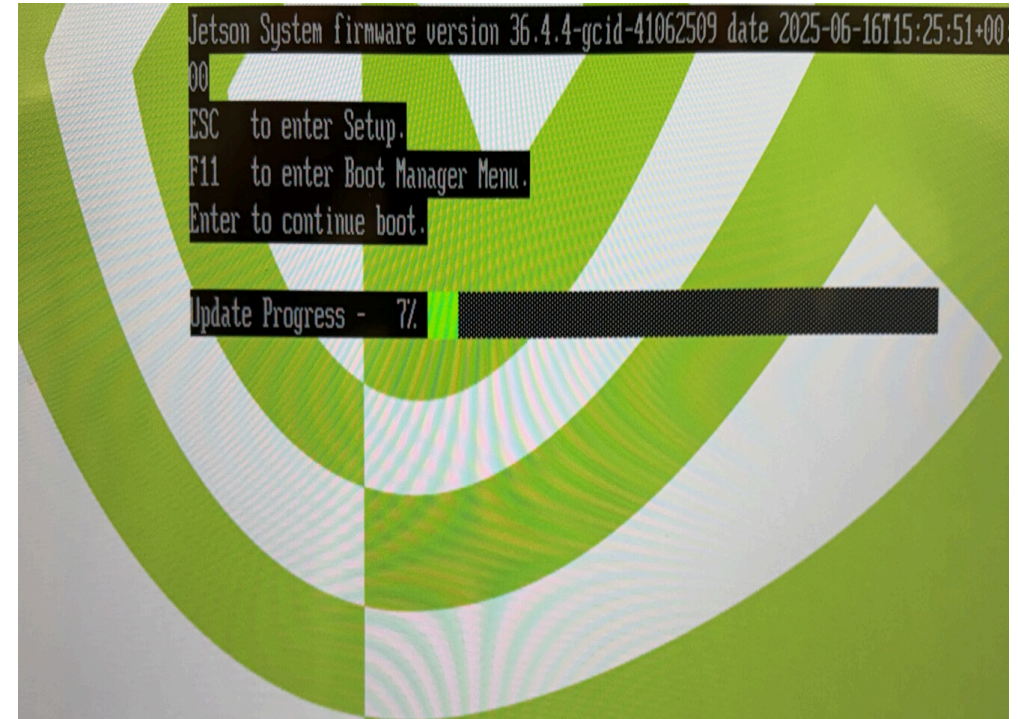
Selected software stack

- nixpkgs nixos-26.05
- anduril/jetpack-nixos
- JetPack 7.2.1, L4T 39.2.1, CUDA 13.2

First firmware upgrade



Jetsons from random shops have random firmware versions



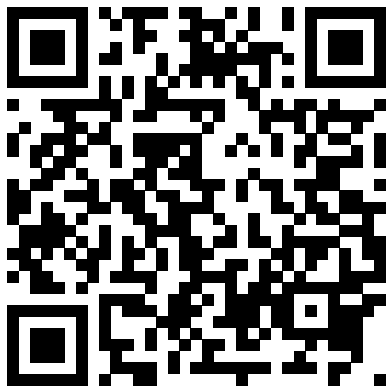
Auto updater from official Jetson image

https://docs.nvidia.com/jetson/orin-nano-devkit/user-guide/latest/quick_start.html

NVIDIA's JetPack SDK, packaged for NixOS

- ▶ packaged components from NVIDIA's JetPack SDK, also inspired from <https://github.com/OE4T>
- ▶ NVIDIA's Linux kernel (incl. nvgpu)
- ▶ EDK2-based UEFI firmware, ARM Trusted Firmware / OP-TEE
- ▶ firmware flashing scripts
- ▶ CUDA, cuDNN, TensorRT
- ▶ HW video encode/decode (V4L2, GStreamer)
- ▶ graphics: Wayland, GBM, EGL, Vulkan
- ▶ power/fan control: nvpmode1, nvfancontrol

device	JP 5	JP 6	JP 7
Thor AGX			✓
Orin AGX	✓	✓	✓
Orin NX	✓	✓	✓
Orin Nano	✓	✓	✓
Xavier AGX	✓		
Xavier NX	✓		



A NixOS USB stick for bootstrapping

- ▶ anduril/jetpack-nixos already contains some minimal ISO image outputs
- ▶ Easy to build your own w/ custom settings:
`applicative-systems/embedded-workshop-jump-game#usbImage`

Optional: install to the much faster NVMe disk

- ▶ disko declares partitions + filesystems (`hosts/nvme.nix`)
- ▶ Same system as the USB image, just a different disk module:
`applicative-systems/embedded-workshop-jump-game#nixosConfigurations.nano`

Where/how to build

- ▶ Cross-compilation: Easy **without** CUDA
- ▶ With CUDA: Must build on ARM with a lot of RAM!
 - ▶ (PR is coming soon that enables cross compilation with CUDA)
- ▶ binfmt based cross-compilation possible but super slow
- ▶ I used my NVIDIA DGX Spark. Relatively quick builds!
- ▶ We revisit image cross compilation in part 2

From our blog: Building Linux from macOS

With enough RAM, you can also build it on macOS!



Build and Deploy Linux Systems from macOS [🔗](#)

Existing CUDA binary caches

```
# NixOS config of the builder
nix.settings = {
  extra-substituters = [
    "https://cache.nixos-cuda.org"
    "https://cache.flox.dev"
  ];
  extra-trusted-public-keys = [
    "cache.nixos-cuda.org:74DUi4Ye579gUqzH4ziL9IyiJBldPMRn9MBN8oNan9M="
    "flox-cache-public-1:7F40yH7ZCnFhczefJdfyXYLQw/aV7GEed86nQ7Is0s="
  ];
};
```

```
# flake.nix: Add here to have flake suggest caches to users
nixConfig = {
  extra-substituters = [
    // ...
  ];
  extra-trusted-public-keys = [
    // ...
  ];
};
```

- ▶ `cache.nixos-cuda.org`: nixpkgs CUDA team
 - ▶ x86_64 only
 - ▶ unstable + stable
- ▶ `cache.flox.dev`: Flox, NVIDIA-sanctioned redistribution
 - ▶ x86_64 + aarch64
 - ▶ revisions of Flox's nixpkgs fork

Limitations

- ▶ both build the **default** `cudaCapabilities`
- ▶ Jetson 8.7 is not a default
 - **no cache hits**, everything from source
- ▶ own caps, `cudaPackages` pin, overlays
 - different hashes
- ▶ → run your own cache

Getting CUDA apps to run: CUDA in nixpkgs

```
nixpkgs.config = {  
  allowUnfree = true;  
  cudaSupport = true;  
  
  # Jetson not in nixpkgs defaults. No caching!  
  cudaCapabilities = [ "8.7" ];  
};
```

- ▶ Must be set in NixOS config
- ▶ Don't "patch it in" via overlays

cudaPackages version ↔ JetPack generation

cudaPackages version	JetPack generation
11.4 - 12.2	JetPack 5
12.4 - 12.9	JetPack 6
13.0 and up	JetPack 7

jetpack-nixos module derives correct version from `majorVersion`

hardware.nvidia-jetpack

```
hardware.nvidia-jetpack = {  
    enable = true;  
    som = "orin-nano";  
    super = true; # Orin Nano Super devkit (67 TOPS)  
    carrierBoard = "devkit";  
    majorVersion = "7";  
    maxClock = true;  
    firmware.autoUpdate = true;  
};
```

The app's CUDA dependencies

```
jump-game
├── torch 2.11
│   ├── cudnn, cublas, cufft, cusolver, ...
│   ├── magma
│   └── triton
├── tensorrt 10.16 (python)
│   └── cuda13.2-tensorrt
├── opencv 4.13
│   └── cudart, cublas, cufft, npp
└── ultralytics 8.4
    ├── torch, torchvision
    ├── opencv-python
    └── onnxruntime 1.24
        └── cudnn
```

- provided in `cudaPackages_13_2`

Overlays for the Jetson

jetpack-nixos already contains important `jetpack-nixos.overlays.default` overlay
also necessary:

```
final: prev: {  
  # No NCCL on Jetson. transitively pulled via torch.  
  pythonPackagesExtensions = prev.pythonPackagesExtensions ++ [  
    (_: pyprev: {  
      torch = pyprev.torch.override {  
        withNvshmem = false;  
      };  
    })  
  ];  
}
```

Refresher in our blog: Dealing with overlays



Mastering Nixpkgs Overlays: Techniques and Best Practice [🔗](#)

Size improvement

```
# board's M.2 card is a Realtek RTL8822CE
# hardware.enableRedistributableFirmware pulls 1.8 GiB of FW
# These are < 1MB
hardware.firmware = [
  (pkgs.runCommand "rtw8822ce-firmware" { } ''
    install -Dt $out/lib/firmware/rtw88 \
      ${pkgs.linux-firmware}/lib/firmware/rtw88/rtw8822c_fw.bin \
      ${pkgs.linux-firmware}/lib/firmware/rtw88/rtw8822c_wow_fw.bin
    '')
];
```

Boot speed improvement: 33.8s → 20.6s

- ▶ Disable boot menu timeout (3.1s → 0.2s)
- ▶ Don't wait for IP addr (-8.7s)
- ▶ Don't wait for wifi (-3.7s)
- ▶ Load GPU driver earlier (initrd +2.7s, user space -5.2s)

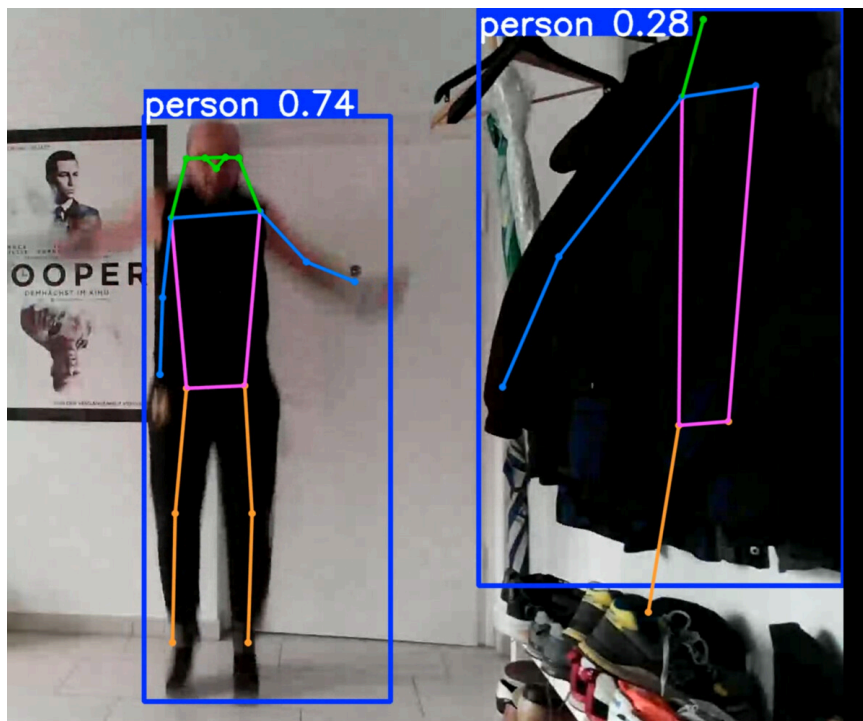
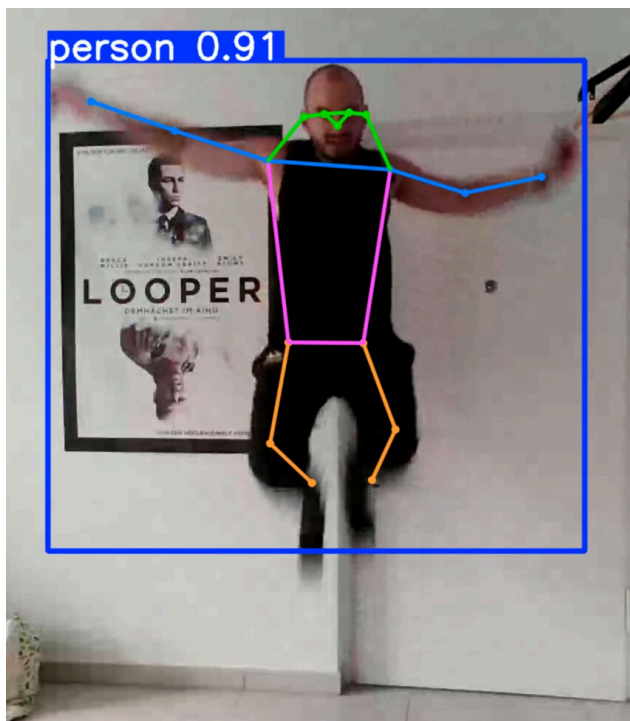
`systemd-analyze critical-chain` was very helpful

Other performance parameters

```
hardware.nvidia-jetpack.maxClock = true;  
services.nvpmode.profileNumber = 2; # MAXN_SUPER: uncapped
```

inference FPS jumped from ~40 to ~50!
(Already including optimization from next slide)

Picking a YOLO pose model



YOLO26	mAP ^{pose} 50-95	T4 TensorRT
n-pose	57.2	1.8 ms
s-pose	63.0	2.7 ms
m-pose	68.8	5.0 ms
l-pose	70.4	6.5 ms
x-pose	71.6	12.2 ms

- Numbers are from the docs
- model sizes between ~8 MB - ~126 MB
- Tradeoff: FPS vs. detection quality

<https://docs.ultralytics.com/models/yolo26>

Inference latency: TensorRT engine files

- ▶ YOLO weights (.pt):
fetched at build time, `fetchurl` + hash
- ▶ TensorRT (.engine):
must be compiled per GPU + driver version +
TensorRT version
- ▶ stale after a TensorRT bump

backend	per frame
eager .pt	44 ms
TensorRT .engine	16 ms

Developing and testing on the desktop

```
# desktop.nix
let
  pkgs = import nixpkgs {
    system = "x86_64-linux";
    config = {
      allowUnfree = true;
      cudaSupport = true;
      cudaCapabilities = [ "7.5" ]; # RTX 2080
      permittedInsecurePackages = [
        "cuda12.9-tensorrt-10.14.1.48"
      ];
    };
    overlays = [ (import ./overlay.nix) ];
  };
in
  pkgs.python3Packages.callPackage ./jump-game { }
```

Build and run with this configuration on your desktop computer for development and testing

NixOS modules: platform specific vs. app specific

```
# somewhere in flake.nix

myNanoConfig = lib.nixosSystem {
  modules.base           # users, SSH, keys, firewall, ...
  modules.jetson         # Jetson-specific settings
  modules.app            # jump game specific service etc.
  modules.size-reduction # dropping stuff for size
  modules.boot-speed     # improve boot process
  modules.usb # or modules.nvme  pick disk medium
}
```

Composes:

Easy to test! Develop and test app module on a normal PC! Swap parts!

Coming from Yocto or JetPack Ubuntu?

	Yocto / meta-tegra	NixOS / jetpack-nixos
board support	BSP layer	<code>hardware.nvidia-jetpack</code> module
product config	image recipe + <code>.bbappends</code>	<code>modules</code> + <code>mkForce</code>
patch a package	<code>.bbappend</code>	overlay
app + OS	different worlds	same language, same repo
dev machine	SDK / container	same flake, <code>nix develop</code>

JetPack Ubuntu: Big, mutable, apt drift per device.

Part 1: links

Jetson jump game demo



[https://github.com/applicative-systems/
embedded-workshop-jump-game](https://github.com/applicative-systems/embedded-workshop-jump-game)

- jetpack-nixos
<https://github.com/anduril/jetpack-nixos>
- Orin Nano devkit quick start
[https://docs.nvidia.com/jetson/orin-nano-devkit/user-guide/
latest/quick_start.html](https://docs.nvidia.com/jetson/orin-nano-devkit/user-guide/latest/quick_start.html)
- Jetson Linux r39.2.1 developer guide
[https://docs.nvidia.com/jetson/archives/r39.2.1/DeveloperGuide/
index.html](https://docs.nvidia.com/jetson/archives/r39.2.1/DeveloperGuide/index.html)

PART 2

**A/B PARTITIONS WITH OTA
UPDATES**

Demo repo - used throughout part 2

A/B immutable partition layout OTA demo



<https://github.com/applicative-systems/nixos-appliance-ota-update>

Demo: an OTA update

```
laptop$ nix run          # VM boots appliance v1

# inside the VM:
parted -l                # B partition: "_empty"
updatectl list           # current version: 1
updatectl check          # server offers version 2
updatectl update         # writes B + new UKI
reboot                  # boot menu: v1 or v2
parted -l                # B partition: "nix-store_2"
systemd-sysupdate vacuum -m 1 # optional
```

- ▶ demo VM serves its own update:
lighttpd → update-v2
- ▶ version is painted into the wallpaper
- ▶ production: systemd-sysupdate.timer, e.g.
daily/nightly in the background
- ▶ vacuum just for show:
v3 would overwrite v1 anyway

Image size reduction: Example workflow

- ▶ desktop appliance without size-reduction.nix: **~1.5 GB** → later **~360MB**
- ▶ loop:
 1. measure
 2. pick unwanted big path
 3. find out: who sets it?
 4. fix
 5. goto 1

```
$ nix build .#image && du -sh result/image.raw
$ TOP=$(nix build --print-out-paths \
    .#nixosConfigurations.appliance.config.system.build.toplevel)
$ nix-store --query --requisites $TOP | xargs du -sm | sort -n | tail # biggest paths
$ nix why-depends $TOP /nix/store/...-source # who pulls it in?
$ nix repl # analyze NixOS module settings
nix-repl> :lf .
nix-repl> :p nixosConfigurations.appliance.options.services.speechd.enable.definitionsWithLocations
```

Noteworthy tool: nix-tree

```
li...-linux-6.1 143.97 MiB (143.97 MiB) ✓ glibc-2.42-84 35.97 MiB (33.44 MiB) ✓ glibc-2.42-84 35.97 MiB (33.44 MiB)
✓ systemd-261.2 181.85 MiB (70.72 MiB) ✓ openssl-3.6.4 44.85 MiB (8.88 MiB) ✓ libgpg-error-1.6 36.83 MiB (886.62 KiB)
system-path 574.65 MiB (64.89 MiB) ✓ p11-kit-0.26.2 41.86 MiB (5.7 MiB)
initrd-linux-6.18.52 34.95 MiB ✓ linux-pam-1.7.2 56.43 MiB (5.38 MiB)
✓ glibc-2.42-84 35.97 MiB (33.44 MiB) ✓ tpm2-tss-4.2.0 69.52 MiB (4.46 MiB)
✓ linux-6.18.52 21.7 MiB ✓ elfutils-0.195 68.02 MiB (4.07 MiB)
✓ uti
-why-depends-
✓ bas nixos-system-appliance-26.11.20260916.b1b8759 → etc-metadata.erofs → dbus-1 → system-path → plymouth-26.134.222
✓ sha nixos-system-appliance-26.11.20260916.b1b8759 → etc-metadata.erofs → dbus-1 → system-path → systemd-261.2 → libg
✓ gli nixos-system-appliance-26.11.20260916.b1b8759 → etc-metadata.erofs → dbus-1 → system-path → systemd-extratools →
✓ cor nixos-system-appliance-26.11.20260916.b1b8759 → etc-metadata.erofs → dbus-1 → system-path → dbus-broker-37 → sys
✓ uti nixos-system-appliance-26.11.20260916.b1b8759 → etc-metadata.erofs → user-units → unit-systemd-tmpfiles-setup.se
✓ fin nixos-system-appliance-26.11.20260916.b1b8759 → etc-metadata.erofs → user-units → unit-nixos-activation.service
✓ bas nixos-system-appliance-26.11.20260916.b1b8759 → etc-metadata.erofs → user-units → unit-nixos-activation.service
✓ gnu nixos-system-appliance-26.11.20260916.b1b8759 → etc-metadata.erofs → user-units → systemd-261.2 → libgcrypt-1.12
✓ nix nixos-system-appliance-26.11.20260916.b1b8759 → etc-metadata.erofs → user-units → unit-dbus.service → systemd-26
✓ kmo nixos-system-appliance-26.11.20260916.b1b8759 → etc-metadata.erofs → user-units → unit-dbus-broker.service → X-R
✓ get nixos-system-appliance-26.11.20260916.b1b8759 → etc-metadata.erofs → user-units → unit-dbus-broker.service → X-R
etc nixos-system-appliance-26.11.20260916.b1b8759 → etc-metadata.erofs → user-units → unit-dbus-broker.service → X-R
etc nixos-system-appliance-26.11.20260916.b1b8759 → etc-metadata.erofs → user-units → unit-dbus-broker.service → X-R
mou nixos-system-appliance-26.11.20260916.b1b8759 → etc-metadata.erofs → user-units → unit-dbus-broker.service → sys
fir nixos-system-appliance-26.11.20260916.b1b8759 → etc-metadata.erofs → user-units → dbus-broker-37 → systemd-261.2
lib nixos-system-appliance-26.11.20260916.b1b8759 → etc-metadata.erofs → pam.d → systemd-261.2 → libgcrypt-1.12.2-li
loc nixos-system-appliance-26.11.20260916.b1b8759 → etc-metadata.erofs → plymouth-themes → plymouth-26.134.222 → sys
etc nixos-system-appliance-26.11.20260916.b1b8759 → etc-metadata.erofs → plymouth-26.134.222 → systemd-261.2 → libgc
nixos-system-appliance-26.11.20260916.b1b8759 → etc-metadata.erofs → etc-sysctl.d-60-nixos.conf → systemd-261.2
✓ util-linux-minim 38.13 MiB (166.03 KiB)
✓ libxcrypt-4.5.2 36.1 MiB (141.0 KiB)
hdv0na9s9y44gazmf5b8cjd97jb3ing-libgcrypt-1.12.2-lib
NAR Size: 1.82 MiB | Closure Size: 38.66 MiB | Added Size: 1.82 MiB
Signatures: cache.nixos.org-1
Immediate Parents (2): systemd-261.2, gnupg-2.4.9
```

Image size: 1.5 GB → 359 MB

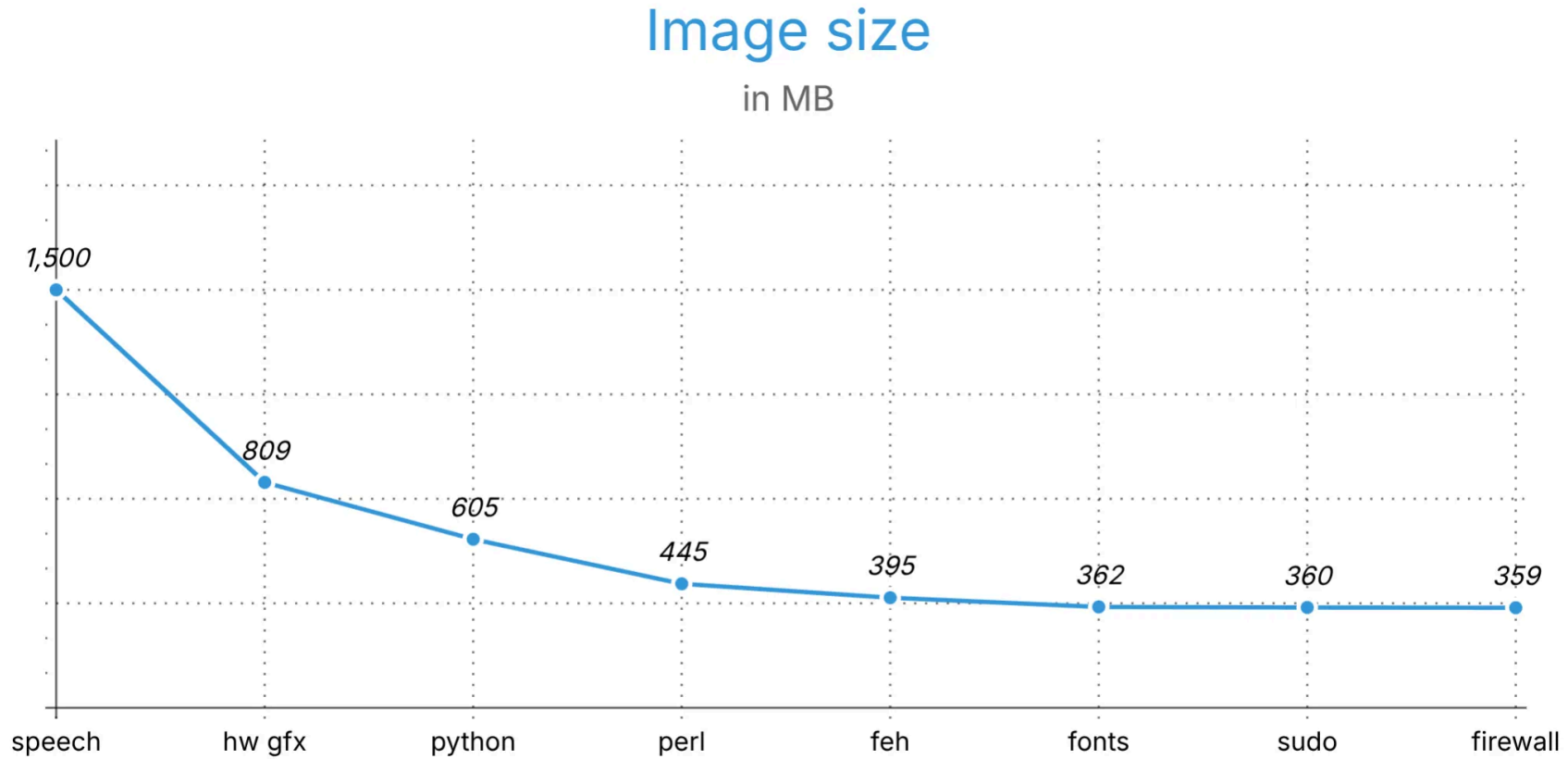


Image size: step by step

change	image
as is	1.5 GB
<code>services.speechd.enable = false</code>	809 MB
<code>hardware.graphics.enable = false</code>	605 MB
iproute2 overlay (Python only for route1)	605 MB
<code>pipewire, libinput off</code>	464 MB
<code>profiles/perlless.nix, xdg-utils = final.bash</code>	445 MB
<code>feh = final.bash</code>	395 MB
fonts: only DejaVu	362 MB

- ▶ `perlless.nix` makes build **fail** if “perl” is in closure
 - ▶ `system.forbiddenDependenciesRegexes`
 - ▶ temporarily `mkForce [ ]` to debug with `why-depends`
- ▶ overlay a dependency with bash:
crude, but “good enough” if it still works
- ▶ test every step: `nix run`

How to get into this how it works?

- ▶ <https://www.freedesktop.org/software/systemd/man/latest/index.html>
- ▶ systemd docs are (nearly) complete
- ▶ difficult to read: no howto, just reference
- ▶ approach: Identify which doc page is important, read it 5x
- ▶ Experiment with the functionality using NixOS integration tests
 - ▶ great howto-doc for the next colleague
 - ▶ or look at our examples on github
- ▶ ~1 day of invest per complex feature
- ▶ profit for life because systemd is here to stay anyway

Self-inflating minimal images

```
# image.nix: image.repart.partitions
#   esp      = UKI + systemd-boot
#   nix-store = /nix/store

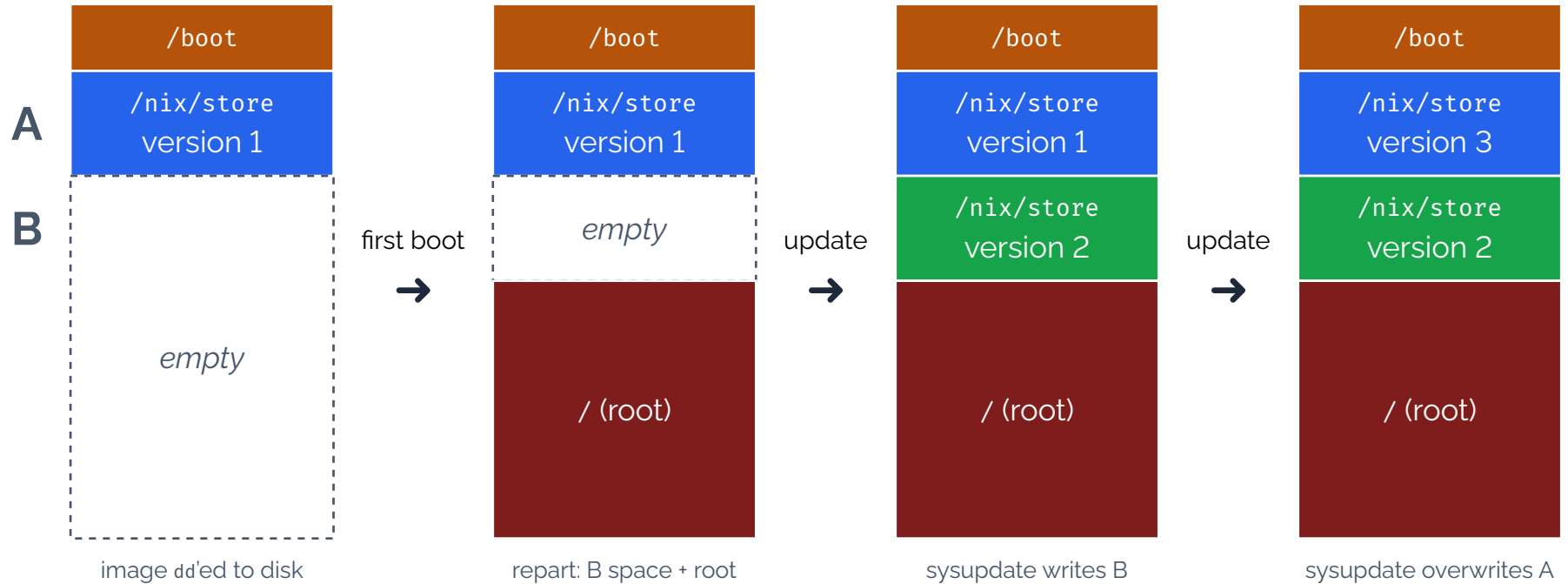
# inflate.nix
boot.initrd.systemd.repart = {
  enable = true;
  device = "/dev/sda";
};
systemd.repart.partitions = {
  home = {
    Type = "home";
    Format = "ext4";
    Weight = 2000;
  };
  var = {
    Type = "var";
    Format = "ext4";
    Weight = 1000;
  };
};
```

test with VM:

```
$ nix build .#image
$ cp result/image.raw demo-disk.raw
$ chmod +w demo-disk.raw
$ qemu-img resize -f raw demo-disk.raw +10G
$ parted demo-disk.raw p # boot, nix-store
$ nix run                # boot the VM
# journalctl -u systemd-repart
# lsblk                  # + home, var
```

- ▶ repart runs in the **initrd**, before anything is mounted
- ▶ repart reconciles disk with config files
- ▶ real disk: `dd if=result/image.raw of=/dev/...`

systemd-repart & systemd-sysupdate in action



A/B partition layout

partition	image.repart.partitions
boot	ESP: systemd-boot + UKIs
nix-store_1	squashfs, read-only, fixed 2 GB (Minimize = "off")
_empty	same size, reserved for system B
root	ext4, persistent state

- ▶ `split = true + SplitName = "nix-store":`
 - ▶ `nix build .#image-v1 → appliance_1.raw (bootstrap) + appliance_1.nix-store.raw (update)`
- ▶ `update-package.nix`: UKI + store image + SHA256SUMS → `nix build .#update-v2`
- ▶ version in file names (`_1`, `_2`): one server folder can host many versions

systemd-sysupdate

```
systemd.sysupdate = {  
  enable = true;  
  transfers."10-nix-store" = {  
    Source = {  
      Path = "https://updates.example.org/";  
      Type = "url-file";  
      # @v matches the version number  
      MatchPattern =  
        [ "${config.system.image.id}_@v.nix-store.raw" ];  
    };  
    Target = {  
      Path = "auto";  
      MatchPattern = "nix-store_@v";  
      Type = "partition";  
      InstancesMax = 2; # A and B  
      ReadOnly = "yes";  
    };  
  };  
  # "20-boot-image": UKI → /EFI/Linux on the ESP  
};
```

1. check server for a newer @v
 2. write store image into the free/oldest partition, relabel nix-store_2
 3. copy new UKI to the ESP, drop old ones
 4. reboot: systemd-boot picks the newest UKI
- demo: Transfer.Verify = "no"

Cross-compilation

```
packages.x86_64-linux.image-aarch64 =  
  (nixosConfigurations.image.extendModules {  
    modules = [  
      {  
        nixpkgs.buildPlatform = "x86_64-linux";  
        nixpkgs.hostPlatform = "aarch64-linux";  
      }  
    ];  
  }).config.system.build.image;
```

```
$ nix build .#image-v1-aarch64  
$ nix run .#vm-demo-aarch64
```

- ▶ `buildPlatform` $\neq$ `hostPlatform` $\rightarrow$ cross build
- ▶ Flakes: `packages.<system>` only picks where Nix **evaluates**, not cross-comp!
- ▶ don't set `nixpkgs.pkgs`: ignores overlays from modules
- ▶ only change `nixpkgs` via `nixpkgs.config`
- ▶ only works if all your code cross-compiles (not CUDA, see part 1)

Outlook: Rugix and delta updates - rough overview

<https://github.com/rugix/rugix>

image.nix

- nix-store_@v + _empty → fixed nix-store-a / nix-store-b
- versioned UKI → one group-agnostic nixos.efi, installed as nixos-a.efi / nixos-b.efi
- static /nix/store mount → initrd generator picks the slot from LoaderEntrySelected

update.nix

- systemd.sysupdate transfers → rugix-ctrl + /etc/rugix/system.toml
- update URL no longer baked into the image

update-package.nix

- files + SHA256SUMS → one .rugixb bundle
- new: rugix-bundler delta OLD NEW OUT → up to ~90 % smaller

Operations

```
# before
updatectl update && reboot

# after
rugix-ctrl update install http://.../update.rugixb
rugix-ctrl system reboot --spare # boot once
rugix-ctrl system commit        # make default
```

- not committed → next boot falls back: automatic rollback
- sysupdate: newest version wins, rollback + vacuum by hand

Unchanged

- systemd-repart, systemd-boot + UKIs, squashfs store, ext4 root, cross-compilation, size reduction

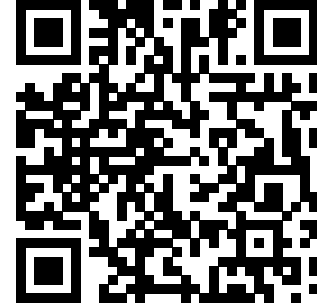
What else from here

- ▶ systemd builtins:
 - ▶ signed updates: `sysupdate.d Verify= (GPG)`
 - ▶ integrity: dm-verity partitions
 - ▶ encryption enrolled at first boot with TPM2/FIDO2: `repart.d Encrypt=`
 - ▶ automatic boot assessment: revert if the new system keeps failing to boot
- ▶ Secure boot
- ▶ System hardening
- ▶ Boot speed improvements

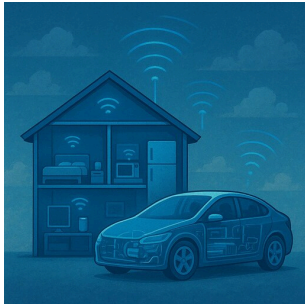
Read more on nixcademy.com



Self-growing NixOS appliance images with
systemd-repart [🔗](#)



Minimizing NixOS images [🔗](#)



NixOS OTA Updates with systemd-sysupdate:
Immutable A/B System Partitions [🔗](#)



Cross-Compiling NixOS images [🔗](#)

Thanks — questions?

Jacek Galowicz

Applicative Systems Group · Nixcademy

- jacek@applicative.systems
- <https://applicative.systems>
-  [tfc](#)  [jgalowicz](#)

